



Alma Mater Studiorum-Università di Bologna  
Scuola di Ingegneria

---

# Installazione del framework JavaFX

*Corso di Laurea in Ingegneria Informatica*

Anno accademico 2023/2024

**Prof. ENRICO DENTI**

*Dipartimento di Informatica – Scienza e Ingegneria (DISI)*



# IL FRAMEWORK JavaFX

- Fino a Java 10, JavaFX era incluso nel JDK
- Da Java 11 è separato e open source: **openjfx.io**

## JavaFX

JavaFX is an open source, next generation client application platform for desktop, mobile and embedded systems built on Java. It is a collaborative effort by many individuals and companies with the goal of producing a modern, efficient, and fully featured toolkit for developing rich client applications.



Download

Home [Documentation](#) Community Testimonials Highlights

## DOCUMENTATION



### Scene Builder

Create beautiful user interfaces and turn your design into an interactive prototype. Scene Builder closes the gap between designers and developers by creating user interfaces which can be directly used in a JavaFX application.

[Wiki](#) [Download](#)



### TestFX

TestFX allows developers to write tests for user interactions and verify the state of nodes.

[Wiki](#) [Repository](#)



### Reference Documentation

- Getting Started with JavaFX 11+
- API documentation
- Introduction to FXML
- JavaFX CSS Reference Guide
- Release Notes



### Community Documentation

- FXDocs
- Jenkov.com
- Almas Baimagambetov's Youtube tutorials



# SCARICARE JavaFX

- Download: <https://gluonhq.com/products/javafx/>
- Disponibile JavaFX 21 (LTS)

gluonhq.com/products/javafx/

GLUON Products Developers Pricing Services

## Downloads

JavaFX version: 21.0.2 [LTS] Operating System: [any] Architecture: [any]

☐ Include older versions

### Supported Platforms

| OS      | Version | Architecture | Type    | Download                                          |
|---------|---------|--------------|---------|---------------------------------------------------|
| Linux   | 21.0.2  | x64          | SDK     | <a href="#">Download</a> <a href="#">[SHA256]</a> |
| Linux   | 21.0.2  | x64          | jmods   | <a href="#">Download</a> <a href="#">[SHA256]</a> |
| macOS   | 21.0.2  | aarch64      | SDK     | <a href="#">Download</a> <a href="#">[SHA256]</a> |
| macOS   | 21.0.2  | aarch64      | jmods   | <a href="#">Download</a> <a href="#">[SHA256]</a> |
| macOS   | 21.0.2  | x64          | SDK     | <a href="#">Download</a> <a href="#">[SHA256]</a> |
| macOS   | 21.0.2  | x64          | jmods   | <a href="#">Download</a> <a href="#">[SHA256]</a> |
| Windows | 21.0.2  | x64          | SDK     | <a href="#">Download</a> <a href="#">[SHA256]</a> |
| Windows | 21.0.2  | x64          | jmods   | <a href="#">Download</a> <a href="#">[SHA256]</a> |
| Javadoc | 21.0.2  |              | Javadoc | <a href="#">Download</a> <a href="#">[SHA256]</a> |

Scaricare la versione SDK  
specifica per la propria piattaforma



# SCARICARE JavaFX

---

- Cosa scaricare?
  - JavaFX è disponibile in due formati: **SDK** e *jmods*
  - **SDK è indispensabile**
  - *jmods* può essere utile
- Cosa sono quei due formati?
  - **SDK** è la libreria vera e propria, da usare per compilare ed eseguire applicazioni, sia da riga di comando che in Eclipse
  - *jmods* è un tipo di archivio per librerie, usato internamente al JDK; si usa insieme al **linker jlink** per assemblare applicazioni che contengano già il supporto per la grafica



# INSTALLARE JavaFX

## JavaFX

### Getting Started with JavaFX

#### Introduction

##### Install Java

##### Run HelloWorld using JavaFX

##### Run HelloWorld via Maven

##### Run HelloWorld via Gradle

##### Runtime images

##### JavaFX and IntelliJ

##### JavaFX and NetBeans

##### JavaFX and Eclipse

##### Next Steps

### Introduction

JavaFX allows you to create Java applications with a modern, hardware-accelerated user interface that is highly portable.

There is [detailed reference documentation for JavaFX](#), and this short tutorial will show you how to write a JavaFX 15 application.

For information on how to run JavaFX applications on [Mobile](#).

For information on Long Term Support (LTS) for JavaFX.

JavaFX builds on top of JDK and is a standalone component for desktop applications:

- Use the [JavaFX SDK](#)
- Use a build system (e.g. maven/gradle) to download the runtime

For both options, it is required to have a recent version of the JDK.

Report a problem

### Getting Started with JavaFX

#### Introduction

##### Install Java

##### Run HelloWorld using JavaFX

##### Run HelloWorld via Maven

##### Run HelloWorld via Gradle

##### Runtime images

##### JavaFX and IntelliJ

##### JavaFX and NetBeans

##### JavaFX and Eclipse

##### Next Steps

### Run HelloWorld using JavaFX

Download an appropriate [JavaFX runtime](#) for your operating system and unzip it to a desired location. For this tutorial, we will be using JavaFX 14.

Add an environment variable pointing to the lib directory of the runtime:

Linux/Mac

Windows

```
set PATH_TO_FX="path\to\javafx-sdk-14\lib"
```

You can now compile and run JavaFX applications from the command line using the JavaFX runtime.

Compile the application (e.g. use [HelloFX.java](#) from this [sample](#)) using:

Linux/Mac

Windows

```
javac --module-path %PATH_TO_FX% --add-modules javafx.controls HelloFX.java
```

**Note:** Additional modules are required for extended functionality. For example, if your application is using `FXML`, you will need to add the `javafx.fxml` module, as shown below:

Linux/Mac

Windows

```
javac --module-path %PATH_TO_FX% --add-modules javafx.controls,javafx.fxml HelloFX.java
```



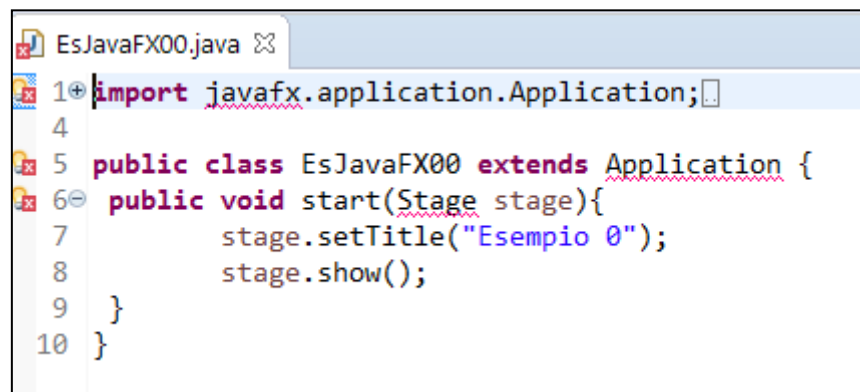
# JavaFX: CONFIGURAZIONE

---

- Dopo aver installato JavaFX
  - per la compilazione e l'esecuzione da riga di comando si devono *specificare opportuni argomenti aggiuntivi da console*
  - per la compilazione e l'esecuzione da dentro Eclipse si deve prima *configurare opportunamente l'ambiente*
- Per configurare JavaFX in Eclipse ci sono due modi:
  - agire volta per volta, progetto per progetto (scomodo)
  - *agire una volta per tutte, definendo un «JRE con JavaFX»* che Eclipse dovrà usare di default da quel momento in poi  
→ *nelle slide seguenti illustreremo questo secondo metodo*
- In mancanza di questa configurazione, non funzionerà nulla
  - Eclipse non troverà JavaFX, dando errori su errori

# JavaFX & Eclipse

- In mancanza di tale configurazione, un progetto che usi JavaFX darà luogo, in Eclipse, a errori di compilazione:



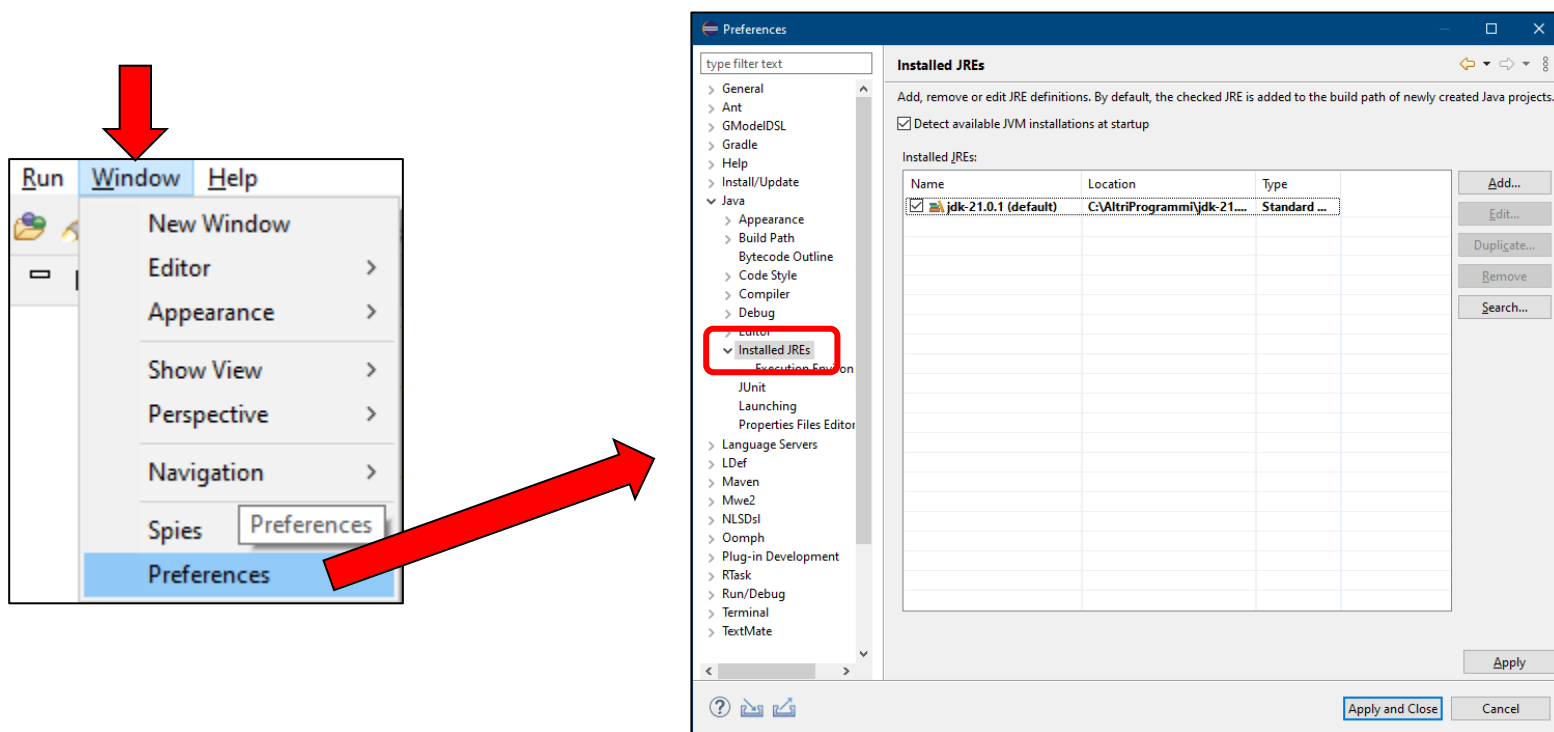
```
EsJavaFX00.java
1+ import javafx.application.Application;
4
5 public class EsJavaFX00 extends Application {
6- public void start(Stage stage){
7     stage.setTitle("Esempio 0");
8     stage.show();
9 }
10 }
```

- MOTIVO: anche se installato, JavaFX non è nel *module path* di default e quindi le relative classi non vengono trovate...
- ...a meno che, come faremo ora, non si predisponga prima un JDK «apposito» che le includa ☺

# JavaFX & Eclipse: configurazione

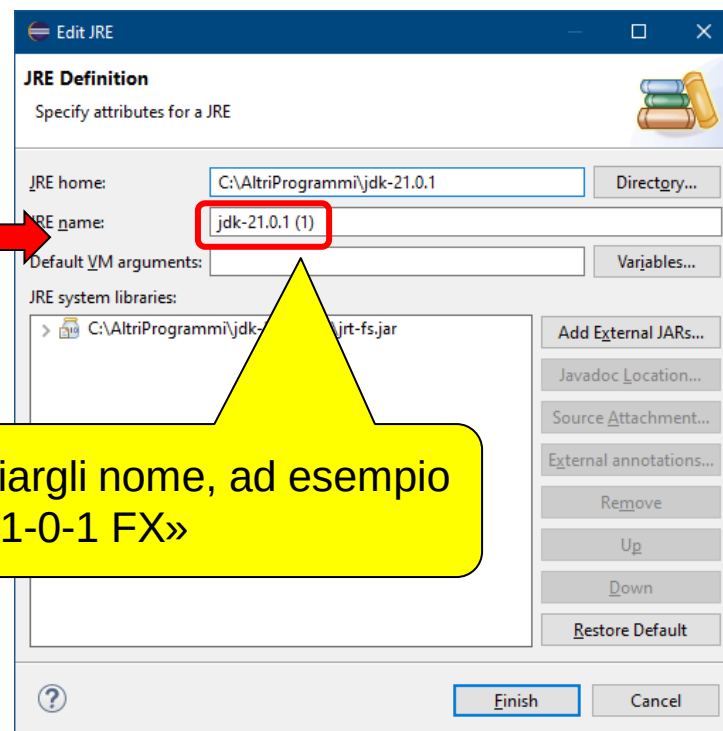
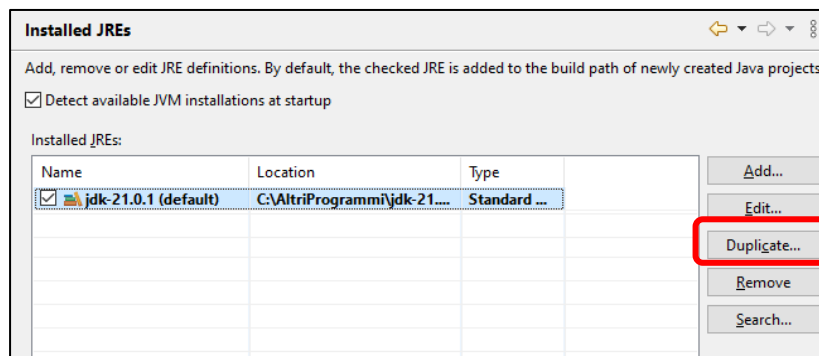
- Per far sì che Eclipse usi JavaFX «di default», occorre **definire un JRE apposito che includa già JavaFX**

1. dal menù Window, scegliere le preferenze di Eclipse e selezionare Java → Installed JREs



# JavaFX & Eclipse: configurazione

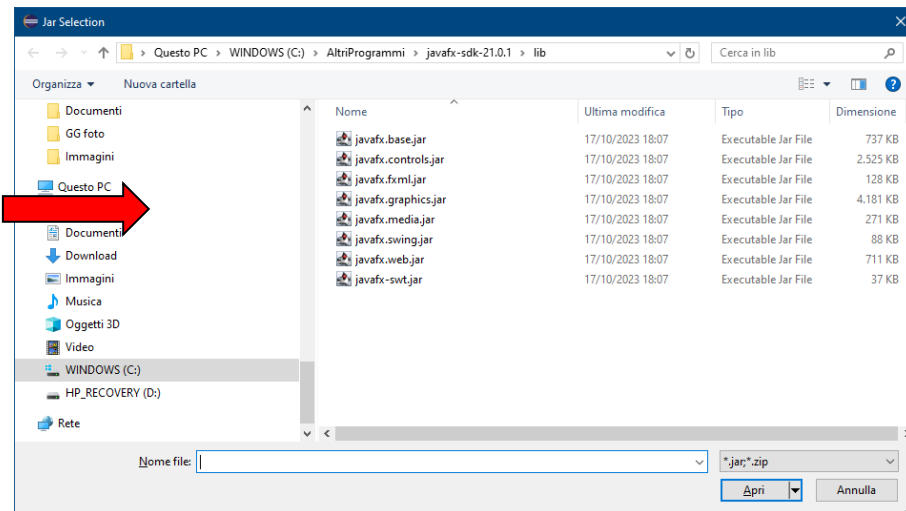
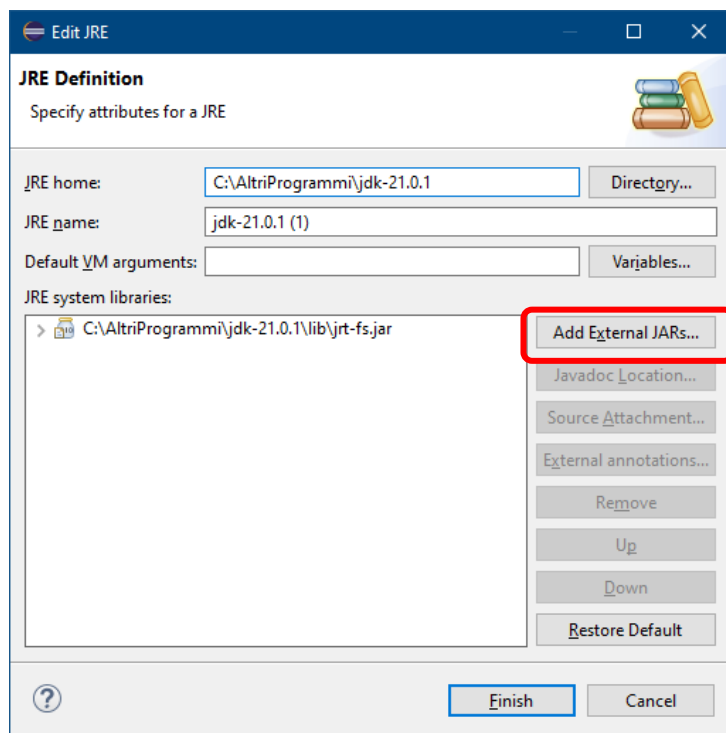
- Per far sì che Eclipse usi JavaFX «di default», occorre definire JRE apposito che includa già JavaFX
2. ora, **selezionare il JRE** che si intende usare e **duplicarlo**



Cambiargli nome, ad esempio  
«jdk-21-0-1 FX»

# JavaFX & Eclipse: configurazione

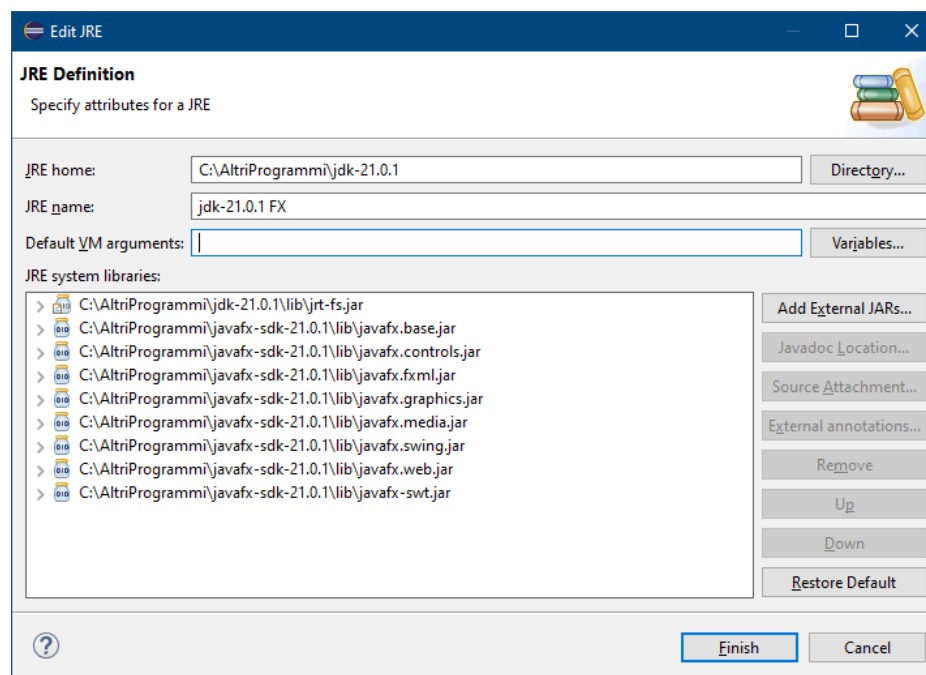
- Per far sì che Eclipse usi JavaFX «di default», occorre definire JRE apposito che includa già JavaFX
3. dopo averlo rinominato, **clicare su «Add External JARs»** e, nella finestra che compare, navigare fino alla cartella di JavaFX\lib





# JavaFX & Eclipse: configurazione

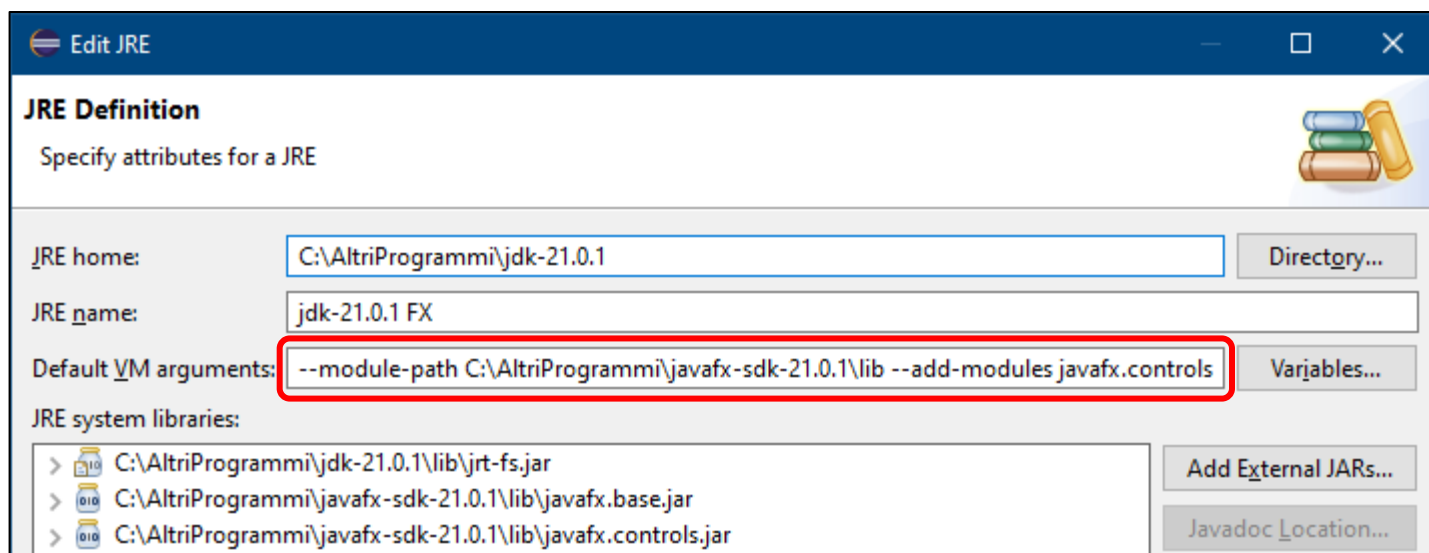
- Per far sì che Eclipse usi JavaFX «di default», occorre definire JRE apposito che includa già JavaFX
  4. ora **selezionare tutti i JAR** e confermare con «Apri»
  5. la situazione sarà questa:



# JavaFX & Eclipse: configurazione

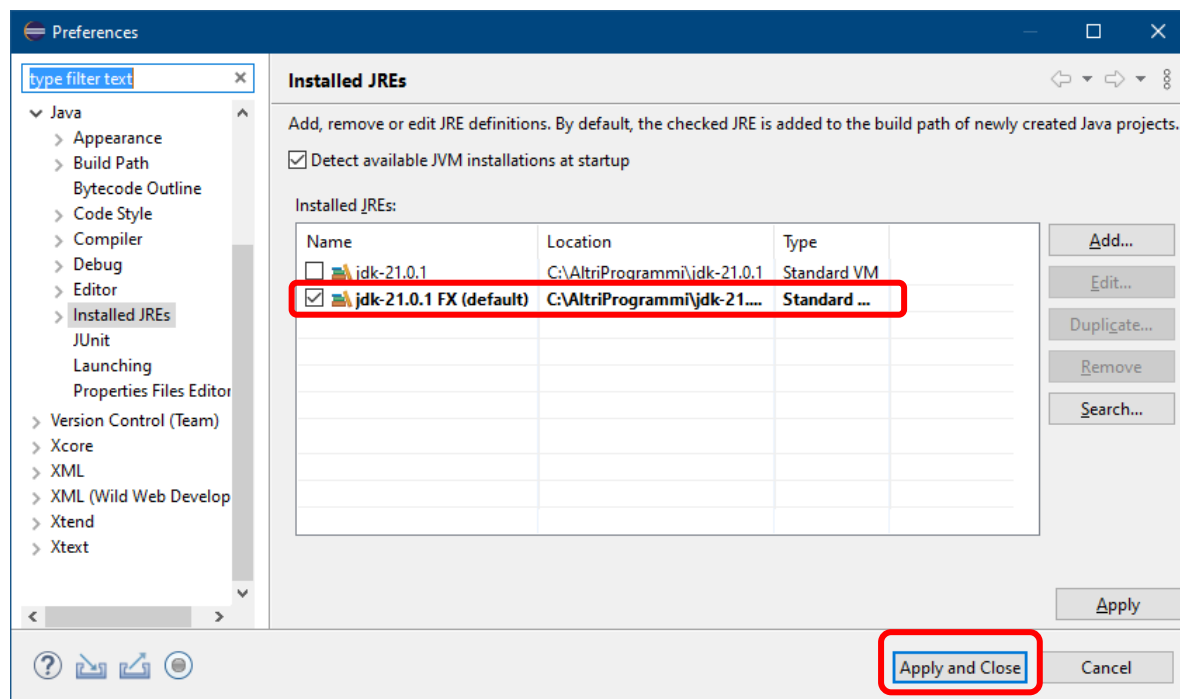
- Per far sì che Eclipse usi JavaFX «di default», occorre definire JRE apposito che includa già JavaFX
6. abbiamo quasi finito: rimangono da **specificare i VM arguments** che diranno a Eclipse dove reperire JavaFX: poi premere Finish

**--module-path ...\lib -add-modules javafx.controls**



# JavaFX & Eclipse: configurazione

- Per far sì che Eclipse usi JavaFX «di default», occorre definire JRE apposito che includa già JavaFX
7. Infine, nella finestra iniziale, **scegliere il nuovo JRE FX come default (diviene in grassetto)** e confermare





# JavaFX & Eclipse: configurazione

---

- Ora Eclipse è configurato e funzionante 😊
- Tutti i progetti nel workspace corrente useranno di default il nuovo JRE selezionato, che include JavaFX
- NB: in altri workspace, o per specifici progetti, potrebbe però essere stato impostato un diverso JRE: quindi, se qualcosa non andasse... verificate! 😊



# Eventualmente..

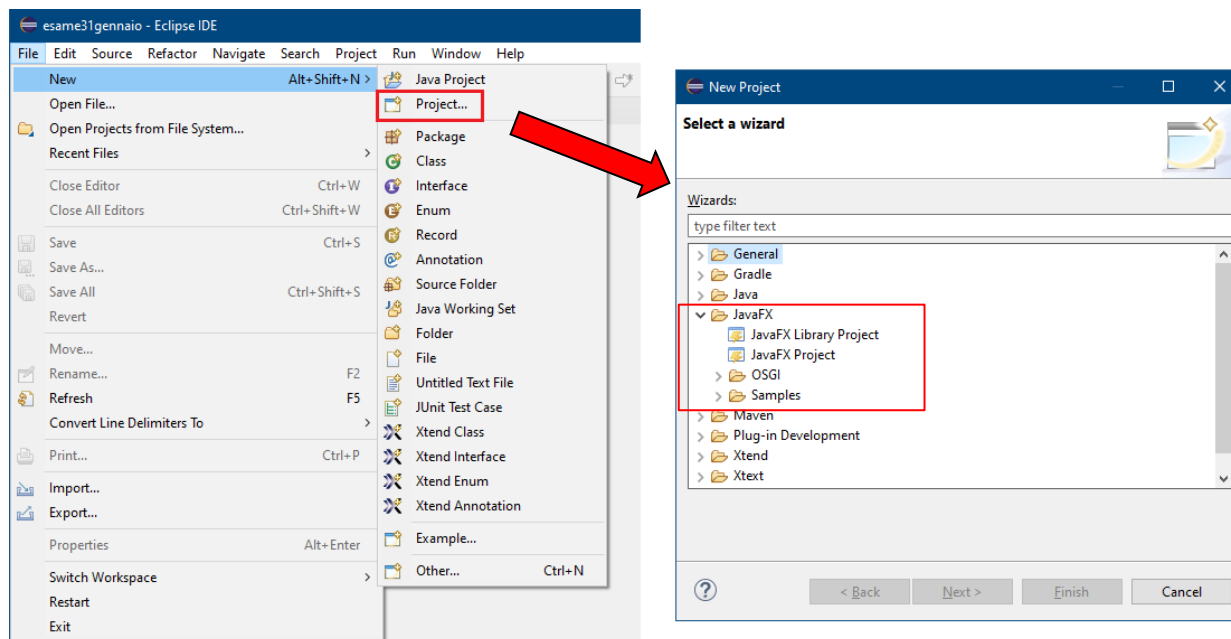
# PROGETTI JavaFX in ECLIPSE

- Di base, un'applicazione JavaFX è, per Eclipse, «soltanto» una normale applicazione Java
- Conseguenze:
  - occorre specificare il main (che in JavaFX sarebbe opzionale)
  - non c'è supporto predefinito per stili, fxml, etc.
- Per gli scopi del nostro corso va benissimo così
  - noi codifichiamo la parte grafica programmaticamente
- Tuttavia, se si desidera un supporto più completo, è possibile averlo installando *plugin e(fx)clipse*
  - ultima versione: 3.9.0 (settembre 2023)

| Topic                                                                               |                                                                                                  |
|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
|  | e(fx)clipse 3.9.0 has been released<br><small>By: Thomas Schindl on Mon, 17 October 2022</small> |

# IL PLUGIN PER ECLIPSE: e(fx)clipse

- Il plugin e(fx)clipse per JavaFX
  - aggiunge il concetto di "JavaFX project", con le sue prospettive
  - genera automaticamente gli scheletri delle classi e dei metodi base
  - evita di dover includere il main "standard"
  - *e altre comode cose* 😊





# IL PLUGIN PER ECLIPSE: e(fx)clipse

---

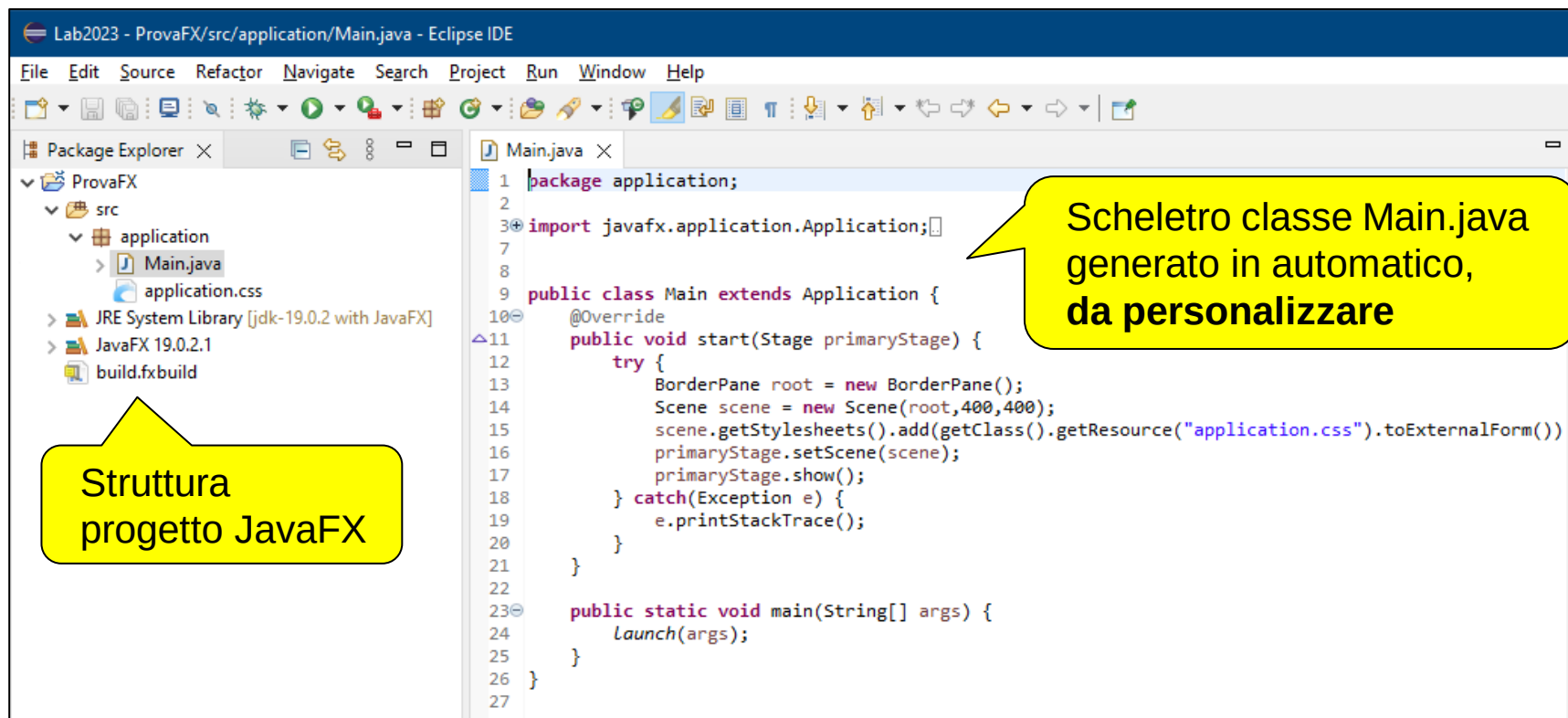
- Il plugin e(fx)clipse per JavaFX
  - aggiunge il concetto di "JavaFX project", con le sue prospettive
  - genera automaticamente gli scheletri delle classi e dei metodi base
  - evita di dover includere il main "standard"
  - *e altre comode cose* 😊

PERÒ:

- non c'è in laboratorio
- è un progetto autonomo da JavaFX, aggiornato con cadenza autonoma → *non diventatene dipendenti*

# ECLIPSE CON JAVAFX

- Questo progetto nasce con la *giusta struttura JavaFX* e include già lo scheletro della classe base e del main



The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure: ProvaFX (src) (application) (Main.java, application.css). The Main.java file is open in the editor, showing the following code:

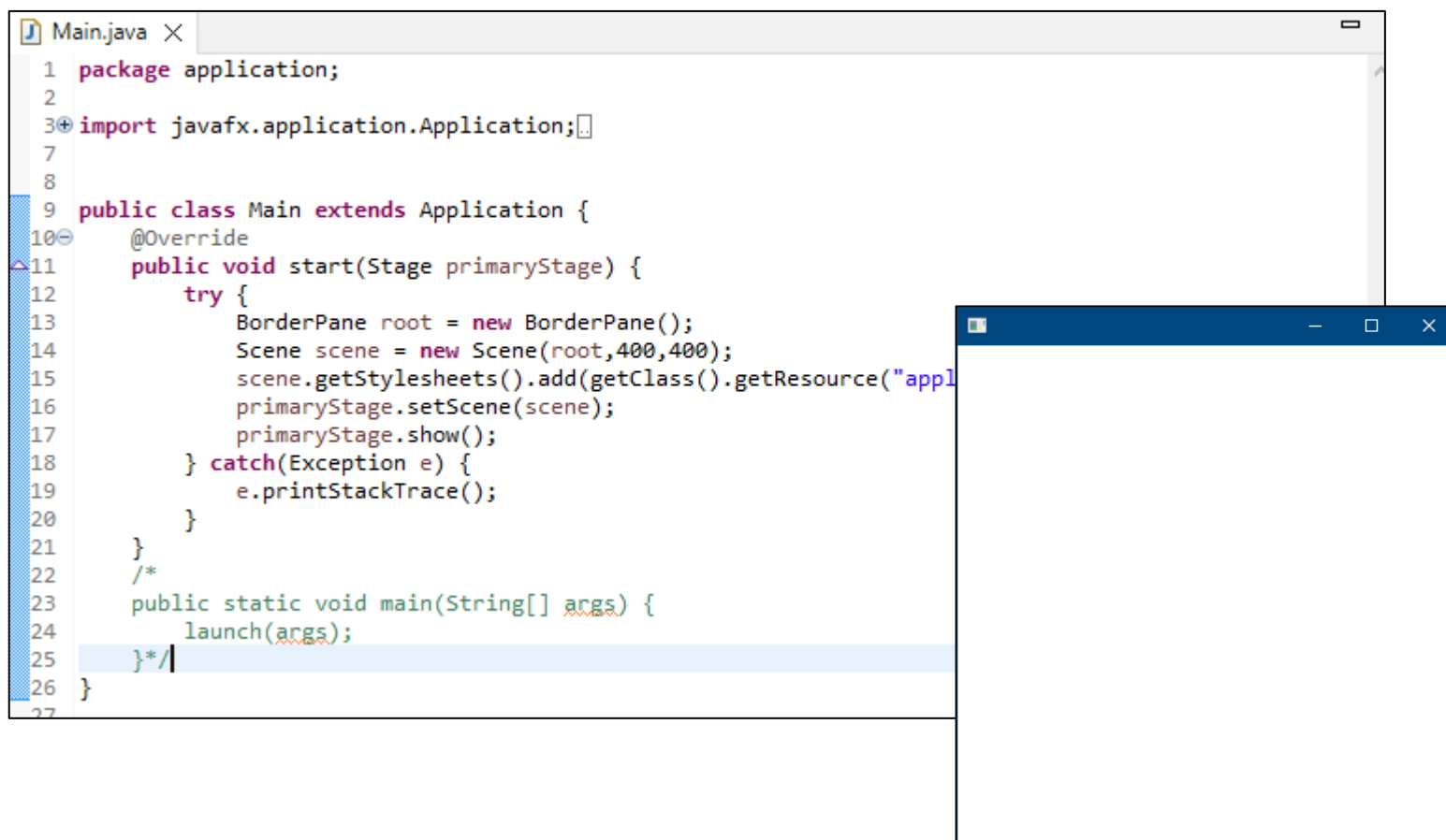
```
1 package application;
2
3 import javafx.application.Application;
4
5
6
7
8
9 public class Main extends Application {
10     @Override
11     public void start(Stage primaryStage) {
12         try {
13             BorderPane root = new BorderPane();
14             Scene scene = new Scene(root, 400, 400);
15             scene.getStylesheets().add(getClass().getResource("application.css").toExternalForm());
16             primaryStage.setScene(scene);
17             primaryStage.show();
18         } catch (Exception e) {
19             e.printStackTrace();
20         }
21     }
22
23     public static void main(String[] args) {
24         launch(args);
25     }
26 }
27
```

Two yellow callout boxes provide additional information:

- Struttura progetto JavaFX**: Points to the Package Explorer showing the project structure.
- Scheletro classe Main.java generato in automatico, da personalizzare**: Points to the code in Main.java, indicating it is an automatically generated skeleton.

# ECLIPSE CON JAVAFX

- Così è possibile eseguire l'app senza specificare il main:

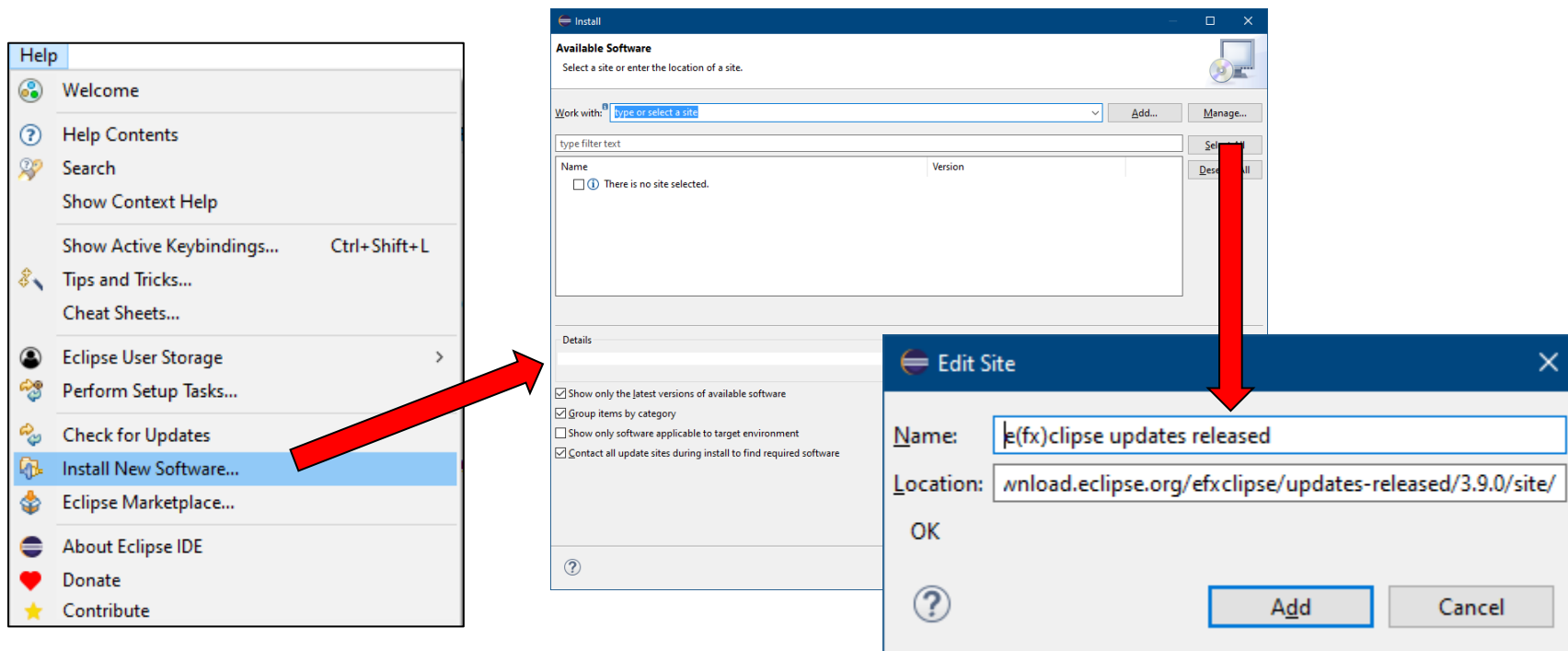


The screenshot shows the Eclipse IDE with a Java file named `Main.java` open. The code defines a `Main` class that extends `Application` and overrides the `start` method. The `start` method creates a `BorderPane` root, a `Scene` with a width of 400 and height of 400, and sets the scene on the `primaryStage`. The `main` method is annotated with `launch(args)` and `launch(args);` is called. The application window is visible on the right, showing a blank white area.

```
1 package application;
2
3 import javafx.application.Application;
4
5
6
7
8
9 public class Main extends Application {
10     @Override
11     public void start(Stage primaryStage) {
12         try {
13             BorderPane root = new BorderPane();
14             Scene scene = new Scene(root, 400, 400);
15             scene.getStylesheets().add(getClass().getResource("appl
16             primaryStage.setScene(scene);
17             primaryStage.show();
18         } catch (Exception e) {
19             e.printStackTrace();
20         }
21     }
22     /*
23     public static void main(String[] args) {
24         launch(args);
25     }*/
26 }
27
```

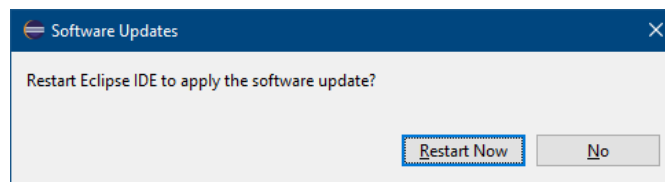
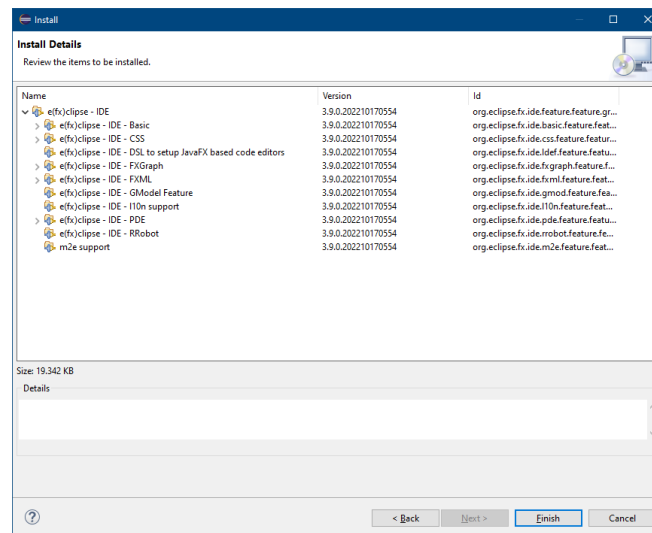
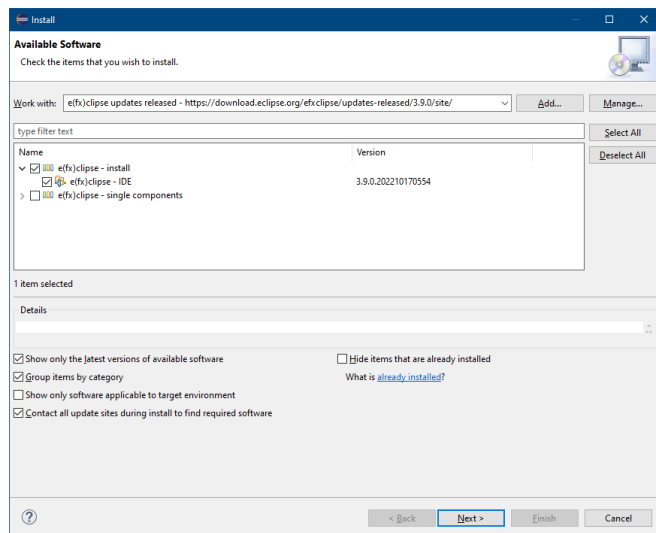
# e(fx)clipse: INSTALLAZIONE

- Eclipse → Help → Install New Software
  - aggiungere un nuovo update site (pulsante «Manage») per e(fx)clipse  
<https://download.eclipse.org/efxclipse/updates-released/3.9.0/site/>



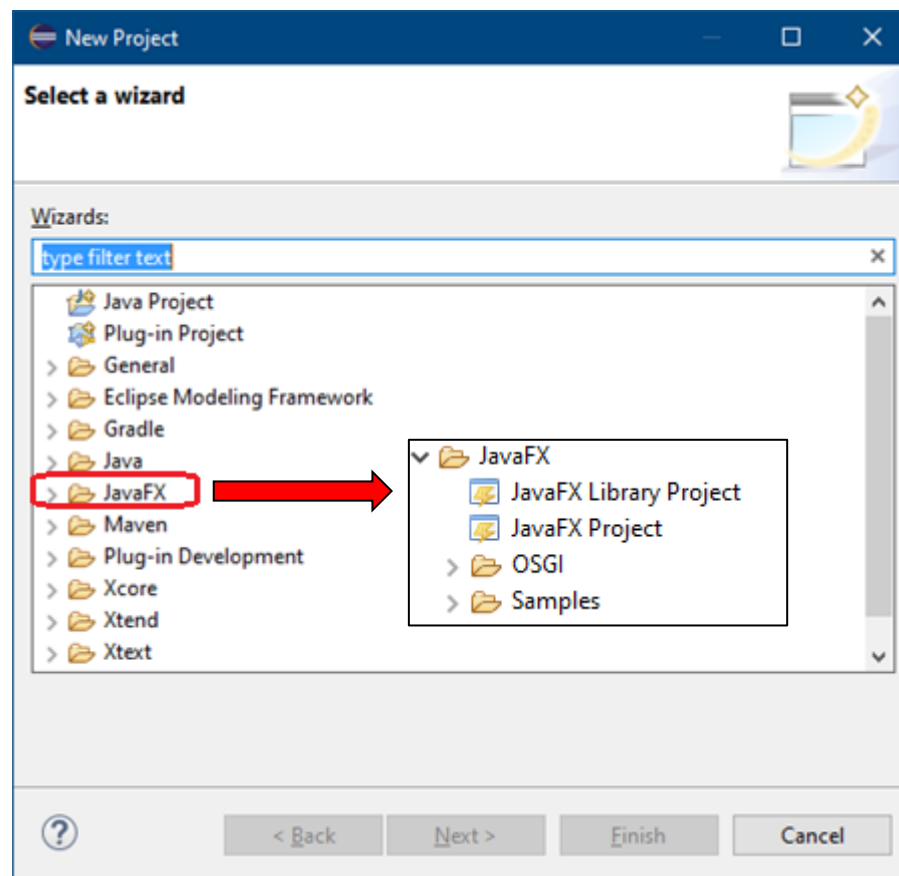
# e(fx)clipse: INSTALLAZIONE

- Ora, nella finestra «available software», selezionare il nuovo update site e cercare e(fx)clipse
  - selezionare l'IDE e confermare (Next) per due volte
  - Eclipse inizierà l'installazione, poi chiederà di riavviarsi:



# e(fx)clipse: INSTALLAZIONE

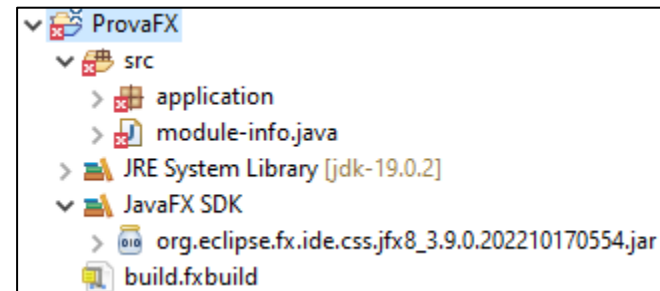
- Se tutto è ok, scegliendo da menù «Project..» dovreste trovarvi queste opzioni:
  - è **cruciale che nell'elenco compaia la voce JavaFX** (se così non fosse, qualcosa è andato storto)



# e(fx)clipse: NUOVO PROGETTO

- Creiamo dunque un nuovo progetto e(fx)clipse, scegliendo da menù «Project..» la voce «**JavaFX project**»

- nome progetto: «ProvaFX»
- Eclipse genererà la struttura:  
il risultato è un nuovo progetto  
(ancora da configurare)
- come prima cosa dobbiamo  
**rimuovere il file module-info.java**



- poi dobbiamo **aggiungere la nostra user library JavaFX** al posto di quell'SDK pre-incluso
- ora il progetto è corretto e si compila senza errori.

